

* Vectors

The greatest strength of R programming is its ability to evaluate fns over an entire vector.

- In R, the most basic data object is the vector.
- A vector is a sequence of elements of the same data type. In other words, vectors are homogeneous.
- There are six types of vectors: logical, integer, double, complex, character, raw.
- The elements of a vector are called its components.
- If we create vector with data of different data types, 'R' forces the data into one mode.

⇒ creating a Vector

⇒ Vectors can be created in different ways, They are most commonly created using the combine fn: , which allows us to combine values into a vector.

a) The combine fn: c() can be used to form a vector.

Ex > apple ← c("red", "green", "yellow")
 > print(apple)
 > print(class(apple))

OP: "red" "green" "yellow"
 "character"

b) The : operator can also be used to create a vector of consecutive numbers

Ex x ← 11:17 ; print(x) OP: 11 12 13 14 15 16 17

(8)

```

ex > y <- 2:-4
    > y
    o/p: 2 1 0 -1 -2 -3 -4
  
```

c) More complex sequences, such as defining the no. of points in a interval or the step size, can be created using the seq() fn:

```

ex > vec = seq(1, 13, by=2) # specify step size
    > vec
    o/p: 1 3 5 7 9 11 13
  
```

```

→ > vec = seq(1, 15, length.out=5) # specifies the length of the vector.
    > vec
    o/p 1.0 4.5 8.0 11.5 15.0 [we get 5 values as length=5]
  
```

⇒ when we add elements, we are reassigning the vector.
i.e we can append the elements to a vector.

```

ex > x <- c([1:10])
    > x
    o/p: 1 2 3 4 5 6 7 8 9 10
  
```

```

    > x <- c(x, 11:15)
  
```

```

    > x
  
```

```

o/p: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
  
```

⇒ we can also name the data elements of a vector.
i.e it's like attaching labels to the vector elements.

```

ex: > vec <- c(10, 11, 12, 13)
     > shapes <- c("spades", "hearts", "diamonds", "clubs")
  
```

> names(vec) ← shapes

> vec

OP: spades hearts diamonds clubs

10

11

12

13

⇒ The min(), max(), which.max() & which.min() fn's are used to find the max & min values of a vector & their position in the vector respectively.

Ex > vec ← c(34, 2, 1, 33)

> vec

OP ¹ ² ³ ⁴
34 2 1 33

> min(vec)

1

> max(vec)

34

> which.max(vec)

1

> which.min(vec)

3

* Accessing the elements of a vector.

If we want to access only some of the vector elements or perhaps an individual element of a vector, we use square brackets [], with vectors & is called indexing a vector. R is very flexible in indexing a vector.

→ we can use a logical, integer or character vector for indexing.

(9)

- Position of the vector elements starts with 1.
- * Passing a vector of positive numbers as indexing elements returns the vector containing the elements at those locations.
- * Passing a vector of negative numbers ~~are~~ as indexing elements returns the vector containing the elements everywhere except at those locations.
- * Passing a logical vector as indexing elements returns the vector containing elements where the index is TRUE.
- * For named vectors, passing a character vector of names as indexing elements returns the slice of the vector containing the elements with those names only.

i) Inter Vector Index

- vector index in R starts from 1. To access a specific vector element, positive/negative indexing can be used.
- vector of integers can ^{also} be used as an index to access specific elements.
- -ve integers used for indexing a vector will return all the elements except those at the specified locations.
- while specifying indexes, we cannot mix positive & negative integer values.
- Real no's used for indexing are truncated to integers.

ex :- > tmp = c(0, 12, 42, 63, 84, 90)

> tmp

o/p: [1] 0 12 42 63 84 90

> tmp[4]

[1] 63

> tmp[c(2, 5)]

[1] 12 84

> tmp[c(2, -4)]

error in tmp[c(2, -4)]

> tmp[c(2.4, 3.5, 4)]

Real nos are truncated to integers.

[1] 12 42

* Logical vector Index

Specifying logical values as index values for accessing vector elements helps us filter through a vector.

→ vector elements at positions where the logical vector specified as indexing is TRUE are returned.

ex > tmp = c(-1, -2, 4, 6)

> tmp[c(TRUE, FALSE, TRUE, FALSE)]

o/p [1] -1, 4

> tmp[tmp < 0] o/p: -1, -2

> tmp[tmp > 0] o/p: 4, 6

* Character Vector Index

Each element of a vector can be named to create an index. When dealing with named vectors, this type of indexing is useful.

Ex `> tmp <- c("first" = 3, "second" = 0, "third" = 9)`

`> names(tmp)`

op: "first" "second" "third"

`> tmp["second"]`

op: second
0

`> tmp[c("first", "third")]`

first third
3 9

⇒ we can assign names to vector values

Ex `> v = c("sam", "Riya")`

`> v`

op: "sam" "Riya"

`> names(v) = c("first", "last")`

`> v`

op: first last
"sam" "Riya"

`> v["first"]`

op: "sam"

* Out of Range vector Index:

→ If an index is out of range, NA will be reported for the missing value

```
> s[10]
```

Op: NA

* Duplicate Index:

→ using duplicate values while indexing in a vector is allowed in R. This results in the retrieval of a member multiple times in the vector.

Ex > tmp <- c("a" "b" "c")

```
> tmp[c(2, 3, 3)]
```

Op: "b" "c" "c"

* Vector Manipulation and vector Arithmetic.

→ A vector can be modified using the assignment operator. Re-assignments can be used to truncate the elements of a vector.

Ex: > v = c(¹1, ²2, ³5, ⁴10, ⁵9)

```
> v[3] = 6
```

```
> print(v)
```

Op: ~~1 2 5~~ 1 2 6 10 9

```
> v[v < 6] <- 8
```

```
> v
```

Op: 8 8 6 10 9

truncating

```
> v <- v[1:4]
```

```
> v
```

Op: 8 8 6 10

here 5th element is truncated.

* Vector Arithmetic

→ vector Arithmetic operations are performed member by member.

Ex > $V_1 = C(1, 3, 5, 7)$

> $V_2 = C(1, 2, 4, 8)$

> $V_1 + V_2$

o/p: [1] 2 5 9 15

> $V_1 - V_2$

o/p: [1] 0 1 1 -1

> $V_1 * V_2$

[1] 1 6 20 56

> V_1 / V_2

[1] 1.000 1.500 1.250 0.875

> $V_1 * 3$

[1] 3 9 15 21

> Prod(V_1)

[1] 105

multiplication of all elements of a vector.

> Sum(V_1)

[1] 16

sum of all elements of a vector.

* Vector Element Recycling

(unequal length)

→ Even if the vectors are of different sizes, we can perform arithmetic operations.

→ when two vectors of unequal length are used for operations, the shorter one will be recycled.

(ie) the values of the shorter vector will be repeated to match the length of the longer vector.

ex: $> v_1 = c(4, 2, 3, 6)$

$> v_2 = c(5, 7)$

#

4	2	3	6
5	7	5	7

$> v_1 + v_2$

o/p: 9 9 8 13

→ we also have `rep()` fn in R programming, that enables the user to create a vector with repeating elements as specified in its parameter.

ex $> \text{rep}(10:12, 3)$

o/p: 10 11 12 10 11 12 10 11 12

$> \text{rep}(10:12, \text{each} = 3)$

o/p: 10 10 10 11 11 11 12 12 12

$> \text{rep}(1:5, \text{length.out} = 9)$

o/p:

1	2	3	4	5	1	2	3	4
1	2	3	4	5	6	7	8	9

(length of vector = 9)

* Combining vectors

→ The `c()` fn: can be used to combine the given vectors.

Ex > `V1 = c(12, 13, 14)`

> `V2 = c("A", "B", "C", "D")`

> `c(V1, V2)`

o/p: `[1] "12" "13" "14" "A" "B" "C" "D"`

here integer values are also forced to

be strings as vectors only hold

elements of the same type.

* Deleting a vector

→ A vector can be deleted by assigning `NULL` to it.

Ex > `V1 = c(4, 5, 6, 1, 2, 3)`

> `V1 <- NULL`

> `V1`

o/p: `[1] NULL`

* vector element sorting

→ The `sort()` fn: can be used to sort the elements of a vector in either ascending or descending order.

→ by default it is ascending order.

Ex > `vec <- c(3, 8, 4, 7, 1, 0, 27)`

> `sort.vec <- sort(vec)` o/p: `0 1 3 4 7 8 27`

> `sort.vec`

→ `vec = c(5, 6, 1, 3, 9, 7, 6)`

→ `result = sort(vec; descending = TRUE)`

→ `result`

O/P: 9, 7, 6, 6, 5, 3, 1

TO get in descending order we need to give `descending = TRUE`

== . ==

Matrices

- * A Matrix is a two-dimensional 'R' object whose elements contain values of the same atomic data type.
 - * Matrices can be created using numeric, character or logical data.
 - * A matrix is also a vector in R, but a vector is not a matrix of either one row or one column.
 - * The basic syntax for creating a matrix in R is :-
`matrix(data, nrow, ncol, byrow, dimnames)`
- where, `data` → refers to the 1D data vector from which a matrix is created.
- `nrow` → no. of rows in the matrix
- `ncol` → no. of columns in the matrix
- `byrow` → is a logical value that indicates how 1D data should be arranged.
- `TRUE` → indicates that the 1D vector elements must be arranged by rows

dimnames → refers to the names of the rows & columns.

ex: > M = matrix(c("a", "b", "c", "d", "e", "f"),
nrow = 2, ncol = 3, byrow = TRUE)

> print(M)

o/p:

	[,1]	[,2]	[,3]
[1,]	"a"	"b"	"c"
[2,]	"d"	"e"	"f"

⇒ TO arrange the data by column:

> M = ~~recoded~~ c(11, 12, 13, 14, 15, 16)

> Mat ← matrix(M, nrow = 3, ncol = 2)

By default fill method is
by column.

> mat

o/p:

	[,1]	[,2]
[1,]	11	14
[2,]	12	15
[3,]	13	16

The default fill method
of the matrix() fn:
is byrow = FALSE.
i.e it fills data by
column.

> class(mat)

o/p: "matrix".

→ Apart from the `matrix()` fn:, there is another easy way to create matrices.

→ * user can paste vectors together using the cbind()
and rbind() fn's.
 ↳ rowbind.

↓
column bind

→ `cbind()` takes the vector passed and sticks them together as if they are columns of a matrix.

→ `rbind()` takes the i/p as rows and makes a matrix out of them.

```
ex > cbind(11:13, 14:16)
```

o/p: $[1, 1]$ $[1, 2]$

[1, 7] 11 14

[2,] 12 15

[3,] 13 16

> rbind(11:13, 14:16)

$[, 1]$ $[, 2]$ $[, 3]$

[1,] 11 12 13

[2,] 14 15 16

```
> data <- c(11, 23, 45, 54, 23, 45)
```

```
> m = matrix(data, byrow = TRUE, nrow = 2)
```

 γ_m

dp: $[,1]$ $[,2]$ $[,3]$

[1,] 11 23 45

[2] 54 23 45

→ bind (m, 7:9)

ok! $[1] [2] [3]$
1 23 45

[1, 3] 11 23 45

[2] 54 23 45

$[3,]$ 7 8 9

* we can also assign names to columns & rows using the `colnames()` & `rownames()` fn's resp.

Ex > m = matrix(11:16, byrow=TRUE, nrow=2)

> m

Op:

	[1]	[,2]	[,3]
[1,]	11	12	13
[2,]	14	15	16

> rownames(m) <- c("row1", "row2")

> colnames(m) <- c("col1", "col2", "col3")

> m

Op:

	col1	col2	col3
row1	11	12	13
row2	14	15	16

* vector can also be converted into a matrix using the `as.matrix()` fn.

Ex > vec <- c(1, 2, 3, 4, 5, 6, 7, 8, 9)

> vec

Op: [1] 1 2 3 4 5 6 7 8 9

> ~~vec~~ mat <- as.matrix(vec)

Op: [1]

[1,]	1
[2,]	2
[3,]	3
[4,]	4
[5,]	5
[6,]	6
[7,]	7
[8,]	8
[9,]	9

> class(mat)
Op matrix

> dim(mat)

Op: 9 1 (9 rows 1 col)

* If ~~the~~ matrix we try to store different data types in a matrix, it automatically forces ^{all} the data to be of one data type.

Ex, `tmp <- c('a', 'b', 'c')`

`> tmp`

o/p: `[1] "a" "b" "c"`

`> t1 <- matrix(tmp, nrow=3)`

`> t1`

o/p: `[,1]`

`[1,] "a"`

`[2,] "b"`

`[3,] "c"`

`> t2 <- matrix(1:6, nrow=3)`

`> t2`

`[,1] [,2]`

`[1,] 1 4`

`[2,] 2 5`

`[3,] 3 6`

`> cbind(t1, t2)`

`[,1] [,2] [,3]`

`[1,] "a" "1" "4"`

`[2,] "b" "2" "5"`

`[3,] "c" "3" "6"`

This is called coercion of Matrix elements

* Matrix Subsetting

If we want to select a single element & entire sections of a matrix, then we use matrix subsetting.

→ E.g. this can be done using rules of square-bracket selection

Ex > m ← matrix(sample(1:15, 12), nrow = 3)

> m

	1	2	3	4
1	1	4	7	10
2	2	5	8	11
3	3	6	9	12

> m[1,3] row 1 col 3

op: 7

> m[3,]

op: 3 6 9 12

> m[,2]

op: 4
5
6

* Matrix operations

⇒ matrix can be modified by applying various operations on it. Some commonly used operations are

* Addition } matrix Algebra.

* Multiplication }

* Transposition → function $t()$ → changes the rows of a matrix to columns & cols to rows.

* Inverse → for this $\det(A) \neq 0$ cond. should be satisfied.

first $\det(A)$ should be calculated. & Apply $inv()$ fn.

* For matrix addition, the two matrices should have same number of rows & columns (matrices should have same shape) otherwise, it produces an error message.

* $\Rightarrow$ The " $*$ " operator is used for element-by-element multiplication of identically shaped matrices.

Ex $\rightarrow t_1 \leftarrow \text{matrix}(1:6, \text{row}=3, \text{ncol}=2)$

$\rightarrow t_1$

Op: $\begin{matrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{matrix}$

$\rightarrow t_2 \leftarrow \text{matrix}(7:12, \text{row}=3, \text{ncol}=2)$

$\rightarrow t_2$

Op: $\begin{matrix} 7 & 10 \\ 8 & 11 \\ 9 & 12 \end{matrix}$

$\rightarrow h \leftarrow t_1 * t_2$

$\rightarrow h$

Op: $\begin{matrix} 7 & 40 \\ 16 & 55 \\ 27 & 72 \end{matrix}$

$\text{mat3} \leftarrow \text{mat1} \% * \% \text{mat2}$
 $\rightarrow \text{mat3 has } m \times p \text{ rows \& cols}$
 $\text{mat1} = m \times n$
 $\text{mat2} = n \times p$

* For matrix multiplication, we use $\% * \%$ operator

which results in a set of sums & cross-products, which require the no. of cols of the first matrix to be equal to the no. of rows of the second matrix.

& resultant matrix has same no. of rows as mat_1 & same cols as mat_2

* special Matrices

(16)

→ one of the special matrix is Diagonal matrix, which has non-zero values along the principal diagonal & 0 everywhere else.

Ex # Diagonal matrix

> D ← diag(c(12, 13, 14), nrow=3, ncol=3)

> D

dp: [1,] [2,] [3,]
 [1,] 12 0 0
 [2,] 0 13 0
 [3,] 0 0 14

→ other special matrix is identity matrix ~~free~~ which is a special form of a diagonal matrix, that has 1 along the principal diagonal & 0 everywhere.

Identity matrix

> Idm ← diag(1, nrow=3, ncol=3)

> Idm

 [1,] [2,] [3,]
dp: [1,] 1 0 0
 [2,] 0 1 0
 [3,] 0 0 1

* Eigen Vectors And Eigenvalues

→ Eigenvectors and eigenvalues are used in machine learning and data preparation.

⇒ R has fn's that can be used to compute the Eigen value and Eigenvectors of a given matrix.

Ex > t1 ← matrix (.5 : 1.5, nrow = 3, ncol = 3)
> t1

Op: [,1] [,2] [,3]

[1,] 0.5 1.5 0.5

[2,] 1.5 0.5 1.5

[3,] 0.5 1.5 0.5

> eigen(t1)

> eigen() decomposition

\$values

[1] 2.886001... 1.3322... -1.38600...

⇒ we can also obtain the eigenvalue & eigenvectors using the eigen() fn:

This returns a named list, ~~with~~ with eigenvalues as the named values & eigenvectors as the named vectors.

Ex > ev ← eigen(t1)

extract components

> values ← ev\$values

> values

Op: [1] 2.8866... 1.3322... -1.3860...

* Data Frames

- R is a statistical programming language where data sets are used.
- Data sets typically comprise observations or instances that have some variables associated with them.
- for ex: consider a dataset of 5 people, where each person such as name, age etc are the variables.
- * A matrix cannot be used to store such information in R, as name would be a character & age is numeric ...
- * So Data frame is a fundamental data structure used to store data sets. it is similar to a spreadsheet that has rows & columns, where each column can ~~be~~ be a different vector. Hence, a data frame can include numbers, characters, text & so on..
- * Thus, it combines the features of vectors, matrices & lists. A data frame contains the same type of data in each of its columns like vectors.
- * But is also two-dimensional like matrices & it can hold a combination of numeric, character & logical data like lists.
- * pictorial representation of various data objects in R:

a) vector



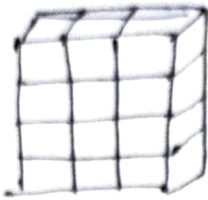
b) Matrix



c) Array



d) Dataframe



e) List {
vectors
Arrays
Dataframes
lists.

* Creating a Data Frame

Data frames are tabular data objects that are two-dimensional like matrices.

→ The columns of a matrix should have the same data type, but in dataframes, one column can be numeric while another can have character values, the next col. can be logical or factor & so on..

* Data frame can also be defined as a list of vectors of the same length.

* The column names & row names should not be empty & should be unique.

* Data frames are created using the data.frame() function.

creating a df.

ex: > BMI <- data.frame(
gender = c("M", "M", "F"),
height = c(152, 171.5, 165),
> Print wt (BMI) = c(81, 93, 78), Age = c(42, 38, 26))

o/p:

	gender	height	wt	Age
1	M	152.0	81	42
2	M	171.5	93	38
3	F	165.0	78	26

Ex: > id <- 1:3

> age <- c(18, 20, 23)

> name <- c("sandy", "puja", "aman")

> data <- data.frame(id, age, name)

> data

o/p:

	id	age	name
1	1	18	sandy
2	2	20	puja
3	3	23	aman.

* General operations on Data frames:

* The str() fn: can be used to view the structure of a data frame.

for Ex: > str(BMI)

'data.frame': 3 obs of 4 variables.

* The number of rows & columns in a dataframe can be obtained by nrow() & ncol() respectively.

* TO print both rows & columns the dim() fn. can be used. It returns the dimension of the object.

* The Summary() fn: gives the nature of the data stored in a dataframe by providing its statistical summary.

i.e. It prints minimum, maximum, mean, median, first quartile & third quartile values for each variable of the dataframe.

* Extracting Data

During visualisation, to ensure that all the data can be seen, we can use head(dataframe) function, which displays the first six rows of the dataframe.

& tail(dataframe) fn: , which displays the last six rows in the dataframe.

* TO subset a dataframe from both matrix & list we can use '[]' single brackets & '[[]' double brackets & '\$' sign.

[] → TO subset data from matrices.

[[]] → } TO select list elements.

\$ →

Ex: result <- data.frame(BMI \$gender)
print(result)

df BMI.gender
M
M
F

Ex

```
> result <- BMI[1:2, ] # extract the
                        first two rows with
                        all columns
> print(result)
```

gender	height	wt	Age
M	152.0	81	42
M	171.5	93	38

* Expanding a Data frame

we can expand a data frame by adding columns & rows to it.

→ TO add a column, new variable, new row or new observation to an existing data frame, the \$ dollar sign or [[]] double square brackets can be used.

Ex > BMI \$ salary <- c(10000, 20000, 30000)

> print BMI

	gender	height	wt	Age	salary
1	M	152.0	81	42	10000
2	M	171.0	93	38	20000
3	F	165.0	78	26	30000

(or)

```
> salary <- c(10000, 20000, 30000)
```

```
> cbind(BMI, salary)
```

→ op -

same op

* Adding a Row

⇒ New rows of the same structure can be added to an existing data frame using the `rbind()` function.

Ex:- `> newBMI <- data.frame(gender = c("F"), height = c(175),
weight = c(94), Age = c(42), salary = c(40000))`

`> finalBMI <- rbind(BMI, newBMI)`

`> print(finalBMI)`

O/P:

	gender	height	wt	Age	salary
1	Male	152.0	81	42	10000
2	Male	171.5	93	38	20000
3	Female	165.0	78	26	30000
4	Female	175.0	94	42	40000

* Merging Data Frames

⇒ Two data frames can be merged based on the column names they have in common, using the `merge()` fn:

`> merge(x, y, by, by.x, by.y, sort = TRUE)`

where `x` & `y` are the data frames to be merged
`by.x` & `by.y` are the specifications of the common columns
& `sort = TRUE` results in sorting by column.

* By default dataframes are merged based on the common column names, but separate column names can be specified in the `by.x` & `by.y` parameters.

The
→ columns comprise the common columns followed by the remaining columns in `x` & then those in `y`.

→ If the matching involves row names, an extra column, Row.names is added to the left & in all cases the result has no special row names.

for ex : here we have 2 data frames teacher & subjects :

```
> teacher <- data.frame(name = c("SA", "SSG", "NA"),
  age = c(55, 35, 45), retired = c("yes", "no", "no"))
```

```
> teacher
```

dp.	name	age	retired
1	SA	55	yes
2	SSG	35	no
3	NA	45	no

```
> subjects <- data.frame(shortname = c("SA", "SA", "SSG", "SSG",
  "NA"),
  teachingSubj = c("Stats", "AI ML", "SSD", "DM", "DBMS"))
```

```
> subjects
```

dp	shortname	teaching subject
1	SA	stats
2	SA	AI ML
3	SSG	SSD
4	SSG	DM
5	<u>NA</u>	DBMS

SA
SA
SSG
SSG
NA

```
> merge(teacher, subjects, by.x = "name", by.y = "shortname")
```

dp	name	age	retired	teaching subject
1	NA	45	no	DBMS
2	SA	55	yes	stats
3	SA	55	yes	AI ML
4	SSG	35	no	SSD
5	SSG	35	no	DM

x y

> merge (subjects, teacher, by.x = "Shortname", by.y = "name")

	Shortname	teaching	age	retired
1	NA	DBMS	45	no
2	SA	stats	55	yes
3	SA	AIML	55	yes
4	SSG	SSD	35	no
5	SSG	DM	35	no

* Applying Functions to Data Frames

⇒ Data frames are special cases of lists, the lapply() & sapply() fn's can be used on dataframes.

⇒ when lapply() is called on a dataframe, each of the frame's columns & return values will be placed in a list.

⇒ The sapply() fn: tries to simplify the output of lapply(), as it returns the whole dataframe after applying the functions specified in sapply().

* lapply() applies functions on individual columns & returns the individual columns of the dataframe.

* whereas sapply() applies functions on the dataframe as a whole & returns the whole data frame.

Ex

```
> lapply(BMI, sort)
$gender
[1] Female Male Male
levels: Female Male
$height
[1] 152.0 165.0 171.5
$weight
[1] 78 81 93
```

```
$Age
[1] 26 38 42
> sapply(BMI, sort)
      gender height weight Age
[1,]      1  152.0    78   26
[2,]      2  165.0    81   38
[3,]      2  171.5    93   42
```

* Factors

Ex
categories - yes/NO
Hair color - (black, white, brown, ...)

ordinal - economic status (low, med, high)
educational exp (elementary school, secondary school, deg, graduate)

24

⇒ There are three kinds of variables in 'R' Programming

- Categorical - variable which has two or more categories (but no intrinsic ordering) → categorical
- ordinal - similar to categorical values with (clear ordering of categories).
- interval - similar to ordinal variable with equal intervals.
Ex: Sal - (5000, 10000, 15000, ...)

⇒ In statistical modelling, categorical variables are those that can take on a finite number of values.

⇒ for this in R, there exists a specific data structure for this, called the Factor.

* Factors deal with non-numeric data values, which can be nominal or categorical.

* creating a Factor

⇒ A categorical variable is one that has categories, but there is no order to the categories.

Ex: Gender is a categorical variable with values Male
Female

Blood type - A, B, AB, O values have ~~no~~ no particular order.

⇒ An ordinal variable is a categorical variable, but there is an intrinsic order to the values that the variable can take.

* * * A factor is a statistical data type that stores categorical variables & is used in statistical modelling.
Factors can be ordered or unordered.

⇒ Factors are stored as a vector of integer values with a set of character values.

⇒ factors are more descriptive & self describing than integers.

factor() fn: is used to create factors.

Ex > t1 <- factor(c("good", "good", "bad", "bad", "nice"))
> t1

o/p: [1] good good bad bad nice good
Levels: bad good nice.

Ex > table(t1)
> t1

o/p: bad good nice
2 3 1

* Factor Levels

→ Factors are very similar to character vectors, but ~~integers are used to store the values~~ integers can be used in factors.

→ Each factor value is associated with an integer, which corresponds to a category or level.

~~Factors do not contain a predefined set of values~~
→ Factors contain a predefined set of values, known as levels which are sorted alphabetically.
i.e if a factor contains 3 values, it has three levels.

Ex: `> review <- factor(c("good", "good", "bad", "nice"))`

R will assign 1 to level bad
2 to good [according to alphabetical order]
3 to nice

→ The `levels()` fn: can be used to check the level of a factor.

Ex `> levels(review)`
[1] "bad" "good" "nice"

`> nlevels(review)` # to find out the number of distinct values.
[1] 3

Ex: `> apple-colors <- c('green', 'green', 'yellow', 'red', 'red', 'red', 'pink')`

`> factor-apple <- factor(apple-colors)`

`> levels(factor-apple)`

`> levels print(nlevels(factor-apple))`

OP levels: green ~~red~~ pink red yellow

[1] 4

* changing the order of levels

→ when a dataset is received, it may contain factors with specific factor levels. But sometimes the names of these levels need to be changed for clarity or for other reasons.

* In R, this task is performed using the levels() fn.
If we apply the factor() fn. again with a new order of levels, we can change the order of the levels in a factor.

Ex:- `> review <- factor(c("good", "good", "bad", "nice",
"bad", "nice", "good"))`

`> review`

Op: [1] good good bad nice bad nice good
levels: bad good nice

Apply factor fn. with required order of levels

`> new-review <- factor(review, levels = c("good", "nice",
"bad"))`

`> new-review`

required order
can be specified.

Op: [1] good good bad nice bad nice good
levels: good nice bad.

⇒ A combination of levels() & labels() can also be used to manually specify arguments when factors are created.

⇒ with levels() - order can be specified.

& with labels() - a new name for the categories can be specified.

Ex `> factor(reviewblondgroup, levels = c("good", "nice", "bad"), labels =
c("bad", "nice", "good"))`

Previous factor

O/p:

good good bad nice bad nice good

now O/p after applying levels & labels

[1] bad , bad good nice good nice bad.

levels : good nice bad.

* Generating Factor Levels

⇒ Factor levels can be generated using the gl() function.
The input for this function is two integers that indicate how many levels ~~are required and how many times each level~~ are required & how many times each ~~level~~ level ~~can~~ be generated.

⇒ Factor levels can be generated using the gl() function
→ The input for this function is two integers that indicate how many levels are required and how many times each level should be generated.

Syntax: gl(a, b, labels)

a → Integer indicating the number of levels.

b → Integer indicating the no: of repetitions.

labels → a vector of labels.

Ex: > review <- gl(3, 2, labels = c("good", "vgood", "nice"))

> review

O/p: [1] good good vgood vgood nice nice

levels : good vgood nice.

* Summarising a Factor

- The generic function summary() can be used on factors as well to produce a summary of the results.
- It gives a quick overview of the contents of a variable.
- The table() function gives similar output as the summary() function for factors.

for ex: `> bloodgrp <- c("B", "AB", "O", "A", "O", "O", "A", "B")`

`> bgfactor <- factor(bloodgrp)`

`> summary(bgfactor)`

o/p: A AB B O
 2 1 2 3

`> table(bgfactor)`

o/p: A AB B O
 2 1 2 3

* ordered Factors

- There are many variables where a natural order exists.
- In the domain of categorical variables, there exists a difference b/w nominal categorical variables & ordinal categorical variables.

ex: `> tshirts <- c("L", "S", "M", "M", "S", "L", "L", "S")`
`> t-factor <- factor(tshirts, ordered = TRUE, levels = c("S", "M", "L"))`
`> t-factor`

o/p: L S M M S L L S
levels : S < M < L

⇒ Also, we can compare ordered factors in R.

```
> tfactor[2] > tfactor[3]
```

FALSE

```
> tfactor[2] < tfactor[3]
```

TRUE

* Converting Factors

→ Factors can be explicitly converted to text or numbers using the `as.character()` or `as.numeric()` fn's.

* Common Functions used with factors

→ `tapply()`, `split()`, & `aggregate()` are commonly used functions with factors.

i) `tapply()` fn: — The `tapply(x, f, g)` fn: has `x` as a vector, `f` as a factor or list of factors, & `g` as a function.

→ It splits `x` into groups formed from factor levels & then the `g()` function is applied on the sub-vectors of `x`.

for ex:

```
> marks <- c(75, 86, 87, 65, 45, 67, 89)
> subjects <- c("Stats", "Stats", "Stats", "DM", "DM", "SSD", "SSD")
> tapply(marks, subjects, mean)
```

o/p

	DM	SSD	Stats
	55.000	78.000	82.666

→ Here, the factor value stats occur at index 1, 2, 3 while factor value DM occurs at 4, 5 & factor value SSD at 6, 7.

→ Groups are formed based on these levels for marks & the mean is calculated.

ii) split() function

→ The `split()` fn: is used to form groups from factors.

The basic form is `split(x, f)` with 'x' being a vector or data frame and 'f' being a factor or a list of factors.

iii) The aggregate() Function :

→ The `aggregate()` fn: calls `tapply()` once for each variable in a group.